第三次周报

本次实验主要研究 PE 文件里 TLS 反调试、IAT HOOK 和 INLINE HOOK。

实验一：TLS 反调试

TLS 是 PE 文件里一个保存局部变量的函数，通过为每一个线程创造备份来处理一个局面：存在一个全局变量和多个线程，A 线程中需要改变全局变量的值，若没有 TLS，则其他线程中的全局变量则会改为 A 线程中设置的值。

源代码如下：

```
#include <Windows.h>
#include <iostream>
_declspec (thread) int g_number = 100;
HANDLE hEvent = NULL;
DWORD WINAPI threadProce1(LPVOID lparam)
{
    g_number = 200;
    printf("g_number = %d\n",g_number);
    SetEvent(hEvent);
    return 0;
}
DWORD WINAPI threadProce2(LPVOID lparam)
{
    WaitForSingleObject(hEvent, -1);
    printf("g_number = %d\n", g_number);
    return 0;
}
int main()
{
    hEvent  = CreateEventA(NULL,FALSE,FALSE,NULL);
    HANDLE hThread1 = CreateThread(NULL, NULL, threadProce1, NULL, NULL, NULL);
    HANDLE hThread2 = CreateThread(NULL, NULL, threadProce2, NULL, NULL, NULL);
    WaitForSingleObject(hThread1, -1);
    WaitForSingleObject(hThread2, -1);
    system("pause");
    return 0;
```

线程 threadProce1 中将 g_number 的值设为 200，在线程 threadProce2 中再输出值仍未 100，运行结果：

```
g_number = 200
g_number = 100
请按任意键继续. . .
```

TLS 反调试的原理：进程启动创建主线程时，先执行 TLS 回调再执行程序入口点（OEP），绕过常规调试器的初始断点。

映射 PE 到内存→加载依赖 DLL→执行 TLS 回调函数→到达程序入口点（OEP）

↑

反调试代码在这里执行

尝试使用 TLS 进行反调试，源代码如下：

```
void NTAPI t_TlsCallBack_A(PVOID DllHandle, DWORD Reason,PVOID Reserve)
{
    printf("TLS function executed\n");
}
//注册TLS函数
#pragma data_seg(".CRT$XLX")
//注册TLS函数.CRT$XLX的作用
//CRT表示使用CRuntime机制
//X表示标识名随机
//L表示TLS Callback section
//X也可以换成B~Y任意一个字符
#pragma data_seg(".CRT$XLX")
//存储回调函数地址
PIMAGE_TLS_CALLBACK pTLS_CALLBACKs[] = {t_TlsCallBack_A, 0 };
#pragma data_seg 0
```

执行结果如下：

```
TLS function executed
TLS function executed
g_number = 200
TLS function executed
TLS function executed
g_number = 100
TLS function executed
请按任意键继续. . .
```

发现 TLS 不止被执行了一次，通过查找资料得知，

#defineDLL_PROCESS_ATTACH 1//进程创建时

#defineDLL_THREADATTACH 2//线程创建时

#defineDLL_THREAD_DETACH 3//线程销毁时

#defineDLL_PROCESS_DETACH 0//进程销毁时

都会执行 TLS

此处了表达简洁，再加上一个 if 判断，让 TLS 在进程创建时输出内容，即：

```
TLS function executed
g_number = 200
g_number = 100
请按任意键继续. . .
```

实验二：IAT HOOK

IAT 和 INT 在程序未被加载到内存时一致；Hook 的本质：劫持执行流。把原本应该调用的函数 A，替换成我们自己的函数 B。

IAT Hook 的步骤：

1.找到目标函数（如 MessageBoxW）在 IAT 表中的位置（指针 g_iatAddr）。

2。保存旧值（原函数地址）：(g_unHookAddr = *g_iatAddr)

3.修改 IAT 表的值：将指向 MessageBoxW 的地址，改成指向我们自己写的 MyMessageBoxW 函数的地址。

先简易写一个 massageboxW，源代码、输出结果如下：

```
#include <windows.h>
int main()
{
    HMODULE hModule =  LoadLibraryA("IAT HOOK.dll");
    if (hModule)
    {
        MessageBoxW(0,L"hello",L"1111",MB_OK);
    }
    return 0;
}
```



再编写 IAT HOOK 的程序

头文件如下：

```
#pragma once
#include <Windows.h>
DWORD* g_iatAdder = NULL;
DWORD* g_unHookAdder = NULL;
int HookMessageBoxW(
    HWND    hWnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT    uType
);
BOOL InstallHook();
BOOL UnstallHook();
DWORD* GetIatAdder
(const char* dllName, const char* dllFunName);
```

主程序分为钩子函数、安装钩子、卸载钩子、主程序和获取 IAT 地址，

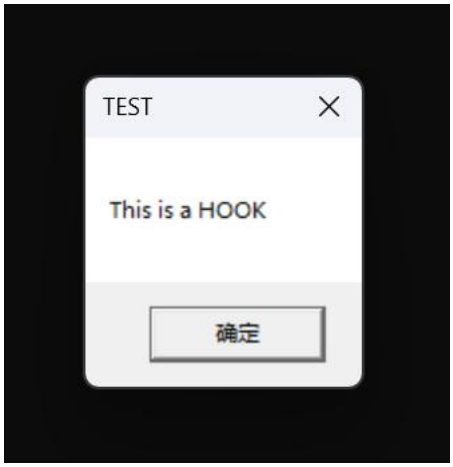
代码依次如下：

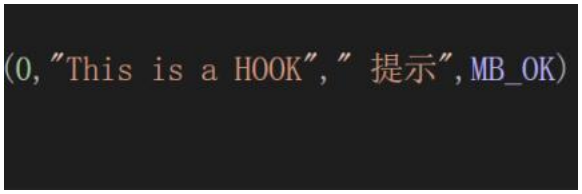
```
#include"IAT_HOOK.h"
int HookMessageBoxW(
    HWND    hWnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT    uType
)
{
    int result = MessageBoxA(0,"This is a HOOK"," 提示",MB_OK);
    return result;
}
//安装钩子
BOOL InstallHook()
{
    DWORD dwOldProce = 0;
    VirtualProtect(g_iatAdder,4,PAGE_EXECUTE_READWRITE,&dwOldProce);
    *g_iatAdder = (DWORD)HookMessageBoxW;
    VirtualProtect(g_iatAdder, 4, dwOldProce,&dwOldProce);
    return true;
}
//卸载钩子
BOOL UnstallHook()
{
    DWORD dwOldProce = 0;
    VirtualProtect(g_iatAdder, 4, PAGE_EXECUTE_READWRITE, &dwOldProce);
    *g_iatAdder = (DWORD)g_unHookAdder;
    VirtualProtect(g_iatAdder, 4, dwOldProce, &dwOldProce);
    return true;
}
```

```
BOOL WINAPI DllMain(HINSTANCE hInstance,DWORD callReason,LPVOID lpReserve)
{
    if (callReason == DLL_PROCESS_ATTACH)
    {
        MessageBoxA(0, "DLL Loaded", "Test", MB_OK);
        //获取IAT表
        g_iatAdder = GetIatAdder("user32.dll", "MessageBoxW");
        //保存HOOK前的函数地址
        g_unHookAdder = (DWORD*)*g_iatAdder;
        //安装钩子
        InstallHook();
    }
    else if (callReason == DLL_PROCESS_DETACH)
    {
        UnstallHook();
    }
    return true;
}
```

```
DWORD* GetIatAdder(const char* dllName,const char* dllFunName)
{
    //获取当前进程中exe文件模块句柄
    HMODULE hModule = GetModuleHandleA(0);
    DWORD dwhModule = (DWORD)hModule;
    //获取DOS头
    PIMAGE_DOS_HEADER pDosHeader = (PIMAGE_DOS_HEADER)dwhModule;
    //获取NT头
    PIMAGE_NT_HEADERS pNtHeader = (PIMAGE_NT_HEADERS)(pDosHeader->e_lfanew + dwhModule);
    //获取可选头
    PIMAGE_OPTIONAL_HEADER pOptionHeader = &pNtHeader->OptionalHeader;
    //获取数据目录表
    IMAGE_DATA_DIRECTORY DataDirectory = pOptionHeader->DataDirectory[1];
    //获取导入表
    PIMAGE_IMPORT_DESCRIPTOR pImportTable =
        (PIMAGE_IMPORT_DESCRIPTOR)(DataDirectory.VirtualAddress + dwhModule);
    //匹配dll文件,遍历导入表寻找符合的要求的函数
    while (pImportTable->Name)//遍历导入表直到结束
    {
        char* iatDllName = (char*)(pImportTable->Name + dwhModule);
        if (_stricmp(iatDllName,dllName) == 0)
        {
            //获取导入名称表
            PIMAGE_THUNK_DATA pINT =
                (PIMAGE_THUNK_DATA)(pImportTable->OriginalFirstThunk + dwhModule);
            //获取导入地址表
            PIMAGE_THUNK_DATA pIAT =
                (PIMAGE_THUNK_DATA)(pImportTable->FirstThunk + dwhModule);
            while (pINT->u1.Function)//同步遍历导入地址、名称表
            {
                if ((pINT->u1.Ordinal & 0x80000000) == 0)
                {
                    PIMAGE_IMPORT_BY_NAME pImportName =
                        (PIMAGE_IMPORT_BY_NAME)(pINT->u1.Function + dwhModule);
                    if (strcmp(pImportName->Name,dllFunName) == 0)
                    {
                        return (DWORD*)pIAT;
                    }
                }
                pINT++;
                pIAT++;
            }
        }
        pImportTable++;
    }
```

通过更改了 IAT 的地址，可以让 Procject1 里面的应该执行的函数地址转化为我们所写的 HOOK 函数地址，达到 HOOK 的效果，结果如下：





与我们所设置的 HOOK 函数相一致（这里由于未知原因中文“提示”输出产生乱码，因此替换为 TEST），实验成功

实验三： INLINE HOOK

IAT HOOK 的原理基于我们的 IAT 表，但当我们需要 HOOK 的函数不在 IAT 表中就没有办法了。HOOK 的本质是在调用某个函数的时候，把该函数的地址替换为我们 HOOK 函数的地址，但函数的地址在内存中会被多次调用，直接修改显然不显示。通过观察汇编指令，发现只需要在函数内部刚加载进来时使用一个 jmp 指令，直接 jmp 到 HOOK 函数的地址的就可以实现一次 HOOK。

前置内容：硬编码，正向代码读写内存的时候无法直接使用 jmp 这个汇编指令，因此需要用到硬编码，jmp 的硬编码为 E9，需要写在代码里的函数地址为：目标地址-（jmp 指令所在地址+5）【即目标地

址-下一个指令的地址】

头文件及源代码如下：

```h
#pragma once
#include <windows.h>
BOOL InitHook();;
int WINAPI MyMessageBoxW(
    HWND    hWnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT    uType
);
BOOL InstallHook();
BOOL UnstallHook();
```

```cpp
#include "main.h"
DWORD g_unhookfun = NULL;
char g_oldcode[5] = {0};
char g_newcode[5] = {0xE9};
//找到我们要HOOK的函数地址
//保存要HOOK的函数的前5个字节
//计算目标函数距离jmp指令的一条指令的编译offset
//改变函数的前5个字节 改成 0xE9 offset
int WINAPI MyMessageBoxW(
    HWND    hWnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT    uType
)
{
    UnstallHook();
    int result = MessageBoxW(0,L"This is a HOOK",lpCaption,uType);
    InstallHook();
    return result;
}
BOOL InstallHook()
{
    DWORD oldProtect = 0;
```

```
    VirtualProtect((DWORD*)g_unhookfun,5,PAGE_EXECUTE_READWRITE,&oldProtect);
    memcpy((DWORD*)g_unhookfun,g_newcode,5);
    VirtualProtect((DWORD*)g_unhookfun, 5, oldProtect, &oldProtect);
    return true;
}
BOOL UnstallHook()
{
    DWORD oldProtect = 0;
    VirtualProtect((DWORD*)g_unhookfun, 5, PAGE_EXECUTE_READWRITE, &oldProtect)
    memcpy((DWORD*)g_unhookfun, g_oldcode, 5);
    VirtualProtect((DWORD*)g_unhookfun, 5, oldProtect, &oldProtect);
    return true;
}
BOOL InitHook()
{
    HMODULE hModule = LoadLibraryA("user32.dll");
    if (hModule == 0)
    {
        return FALSE;
    }
    g_unhookfun = (DWORD)GetProcAddress(hModule, "MessageBoxW");
    memcpy(g_oldcode,(char*)g_unhookfun,5);
```

```
    DWORD offset = (DWORD)MyMessageBoxW - (g_unhookfun + 5);
    memcpy(&g_newcode[1],&offset,4);
    return true;
}
BOOL WINAPI DllMain(HINSTANCE hInstance, DWORD callReason, LPVOID lpReserve)
{
    if (callReason == DLL_PROCESS_ATTACH)
    {
        InitHook();
        InstallHook();
    }
    else if (callReason == DLL_PROCESS_DETACH)
    {
        UnstallHook();
    }
    return true;
}
```

执行代码，结果如下：



实验成功。

四、实验补充与小结

（一）TLS 回调与反调试

补充一下 TLS 在 PE 层面的具体形式：它由数据目录里的 TLS 表描述（索引 9，结构为 IMAGE_TLS_DIRECTORY），表里的 AddressOfCallbacks 字段指向一个以空指针结尾的回调函数数组，数组中的每一项就是一个 TLS 回调，原型大致是 void NTAPI callback(PVOID DllHandle, DWORD Reason, PVOID Reserved)。系统会在进程和线程的创建、销毁这些时间点上调用它们，所以回调有机会比入口点更早执行。

这也解释了截图里回调为什么被执行了多次：进程创建、每个线程创建、每个线程销毁、进程销毁各触发一次，主线程加两个工作线程自然会出现多次输出。加上 Reason == DLL_PROCESS_ATTACH 的判断之后只执行一次。这种手法能绕开初始断点的原因是大多数调试器默认停在系统断点或程序入口点，而 TLS 回调在此之前就跑完了；不过它算不上绝对可靠，调试器可以单独给回调下断点、在选项里跳过系统断点直接跑到 OEP，也可以手工改掉 TLS 目录把回调摘掉。

（二）IAT HOOK 的生效条件与边界

之所以改一张表就能改变程序行为，是因为编译器把对导入函数的调用编译成了“从 IAT 表项里取地址再跳过去”，表项里放的就是函数入口地址，换掉表项，调用就会转到我们写的函数上。另外 IAT 所在的页通常是只读的，直接写会触发访问冲突，所以安装和卸载钩子时都要先用 VirtualProtect 把这几个字节改成可写，改完再恢复原属

性。

（三）INLINE HOOK 的偏移与内存属性

具体到指令层面，E9 后面的 4 个字节是相对偏移，算法是“目标地址 − (jmp 指令所在地址 + 5)”，减 5 是因为 E9 加上 4 字节偏移正好占 5 个字节，而相对量是从下一条指令开始算的。相对偏移只有 4 字节，跳转范围大约在 ±2GB 之内，如果 Hook 函数离得很远，E9 就不够用，需要考虑 FF 25 这类绝对跳转的写法。

还有两个细节需要注意：一是代码段一般不可写，memcpy 之前要先 VirtualProtect 打开写权限，写完再恢复；二是必须把被覆盖的原始字节备份下来，卸载钩子时原样写回，否则后续调用就会跳到已经被改坏的地方。

五、本周感悟

把三个实验放在一起看，它们处理的是同一类问题——改变程序原本的执行路径。TLS 是在入口点之前抢一步，先把自己的代码跑起来；IAT HOOK 是改掉“调用表”里记下的函数地址；INLINE HOOK 则直接动目标函数开头的几条指令，切入点一层比一层底层。

另一个体会是这类实验对细节非常苛刻。TLS 回调为什么执行了那么多次、中文为什么会变成乱码、jmp 的偏移为什么要减 5，这些都不是“差不多对”就能过去的：偏移算错一个字节，程序直接崩，连报错都没有。所以我养成了改之前先备份原始数据、改完立刻恢复的习惯，安装和卸载写成一对，调试前先确认地址再动手。